COS30082 · APPLIED MACHINE LEARNING

Machine Learning Study Guide

Tổng hợp liên kết kiến thức Week 01 → Week 06

Thuật toán · Thuật ngữ · Cách model hoạt động · Điểm cần lưu ý

Tài liệu này dùng để làm gì?

Đây là bản tổng hợp từ 9 file slide (Lecture 1, 2.0, 2.1, 2.2, 2.3, 3, 4, 5, 6). Thay vì học rời rạc từng tuần, tài liệu cho thấy **mỗi tuần nối vào tuần trước như thế nào**: từ một đường thẳng (Linear Regression) → thêm sigmoid (Logistic) → tách lớp bằng lề lớn nhất (SVM) → xếp nhiều neuron thành mạng (ANN) → chuyên cho ảnh (CNN) → tái sử dụng model đã học (Transfer Learning) → vừa phân loại vừa định vị vật thể (Object Detection).

Quy ước: thuật ngữ giữ nguyên tiếng Anh; ký hiệu toán theo slide (θ là tham số, $h_\theta(x)$ là dự đoán, J(θ) là cost). Mục có nhãn **(bổ sung)** là phần mình giải thích thêm cho dễ hiểu, không có nguyên văn trên slide.

- **Phần 0** – Bản đồ tổng thể và "sợi chỉ đỏ" xuyên suốt 6 tuần
- **Week 01** – Nhập môn ML/DL, ML lifecycle, framework, ứng dụng
- **Week 02** – Thuật toán cổ điển: Linear Regression, Logistic Regression, SVM (+ Decision Tree, KNN, K-Means)
- **Week 03** – Deep Neural Network: neuron, forward/backward propagation
- **Week 04** – CNN: convolution, pooling, optimizer, kiến trúc kinh điển, xử lý lỗi huấn luyện, Keras
- **Week 05** – Transfer Learning: feature extractor, fine-tuning, domain adaptation, MTL, zero/one-shot
- **Week 06** – Object Detection: R-CNN → Fast → Faster, YOLO, SSD, IoU, NMS
- **Phần cuối** – Bảng so sánh model, công thức cần nhớ, lỗi hay gặp, glossary, câu hỏi tự kiểm tra

★ Cách học nhanh nhất

• Đọc **Phần 0** trước để nắm bức tranh, sau đó đọc từng tuần theo thứ tự – mỗi tuần mở đầu bằng ô "Nối với tuần trước".

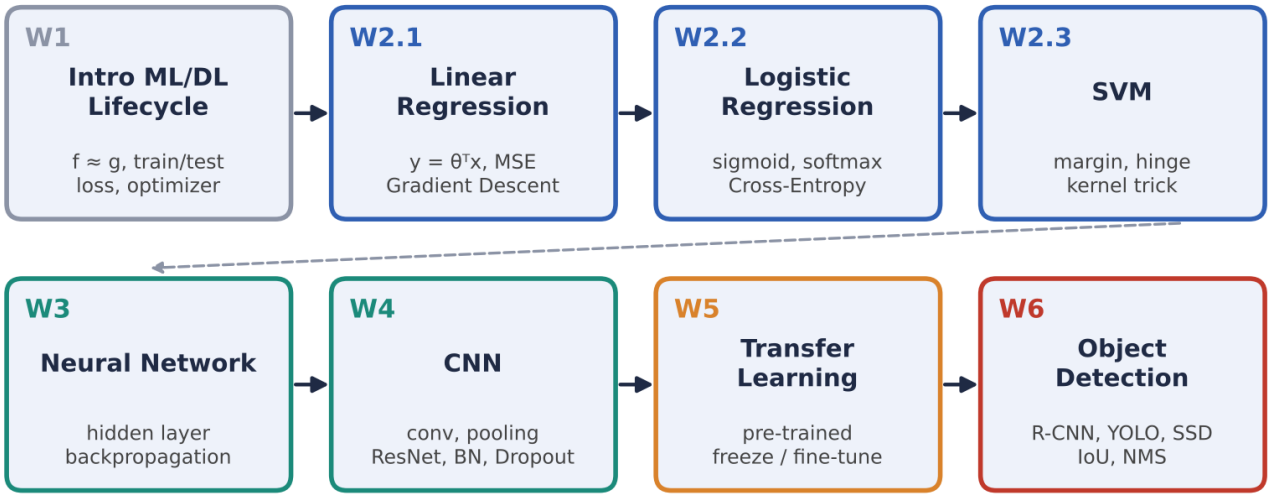
• Đọc ô ⚠ **Sai lầm hay gặp** và ✎ **Mẹo nhớ** ở cuối mỗi tuần – đây thường là chỗ bị hỏi trong bài kiểm tra.

• Cuối tài liệu có **câu hỏi tự kiểm tra** và **cheat sheet công thức** để ôn trước kỳ thi.

P0

Bản đồ tổng thể

Sợi chỉ đỏ nối 6 tuần học



Hình 1 – Lộ trình kiến thức: mỗi khối thêm một "mảnh ghép" vào khối trước.

0.1 Sợi chỉ đỏ: mọi thuật toán supervised đều theo cùng một khuôn

Từ Week 2 đến Week 6, dù model đơn giản hay phức tạp, quy trình học luôn là 4 bước giống nhau (Lecture 2.0 gọi đây là "similar training idea"):

Bước	Ý nghĩa	Linear Reg.	Logistic Reg.	SVM	Neural Net / CNN
1. Giả định model có tham số	Hàm $h_\theta(x)$ với tham số θ (weight + bias)	$\theta^T x$	$g(\theta^T x)$, g = sigmoid	$\theta^T x = 0$ là hyperplane	Nhiều lớp: $g(\theta^{(l)} a^{(l)})$
2. Chọn loss/cost J(θ)	Đo dự đoán sai bao nhiêu	Squared error (MSE)	Cross-entropy	Hinge loss + $\|w\|^2$	Cross-entropy / MSE
3. Tối ưu θ để J nhỏ nhất	Chủ yếu bằng Gradient Descent	Normal Eq. hoặc GD	GD	GD / giải bài toán lề lớn	GD + Backpropagation
4. Dự đoán dữ liệu mới	Đưa x mới qua h_θ	Số thực	Xác suất → lớp	Dấu của z → lớp ±1	Softmax → lớp

0.2 Cái gì "lớn lên" qua từng tuần

Tuần	Thêm gì so với tuần trước	Vấn đề tuần trước chưa giải được
W2.1 Linear Reg.	Học một đường/mặt phẳng khớp dữ liệu liên tục; GD, regularization	Chỉ dự đoán số thực, quan hệ tuyến tính
W2.2 Logistic Reg.	Bọc $\theta^T x$ trong **sigmoid** → xác suất; Cross-entropy thay MSE; softmax cho nhiều lớp	Biên quyết định vẫn phẳng; nhiều biên cùng "tối ưu"
W2.3 SVM	Chọn biên có **lề (margin) lớn nhất**; kernel trick cho dữ liệu phi tuyến	Phải tự thiết kế feature/kernel
W3 ANN	Xếp nhiều logistic unit thành **hidden layer** → mạng tự học feature; backprop	Ảnh có quá nhiều pixel; FC layer tốn tham số, bỏ qua cấu trúc không gian
W4 CNN	**Convolution + weight sharing + pooling**; ReLU, BN, Dropout, ResNet; xử lý vanishing/exploding gradient	Cần rất nhiều dữ liệu và thời gian huấn luyện từ đầu

Tuần	Thêm gì so với tuần trước	Vấn đề tuần trước chưa giải được
W5 Transfer	Tái sử dụng CNN đã train trước (freeze / fine-tune); domain adaptation, MTL, zero/one-shot	CNN chỉ cho 1 nhãn/ảnh, không biết vật ở đâu
W6 Detection	Dùng CNN pre-trained làm **base network** + bounding box regression: R-CNN family, YOLO, SSD	—

✎ **Mẹo nhớ nhanh**

• **Logistic Regression = 1 neuron** (Week 3 nói thẳng: LR là mạng không có hidden layer).

• **CNN = ANN + (convolution, pooling)**; phần cuối của CNN vẫn là FC + softmax như ANN/Logistic multi-class.

• **Detection = Classification (Week 4) + Regression (Week 2.1)**: phân loại *cái gì* và hồi quy *ở đâu* (x, y, w, h). Slide Week 4 cũng nói bài biển số xe có cả regression lẫn classification.

• **Transfer Learning** là cầu nối: R-CNN dùng CNN pre-trained (VGG/ResNet trên ImageNet) rồi fine-tune, và vẫn dùng **SVM** (Week 2.3) để phân loại vùng.

W01

Introduction to Machine Learning

AI ⊃ ML ⊃ DL · lifecycle · framework · ứng dụng

1.1 AI – ML – DL khác nhau thế nào?

Khái niệm	Định nghĩa (theo slide)	Ví dụ
AI	Mô phỏng trí tuệ con người trong máy: giải quyết vấn đề, ra quyết định, hiểu ngôn ngữ, nhận dạng mẫu	Chatbot, xe tự lái
ML (tập con của AI)	Dạy máy **học từ dữ liệu** để dự đoán/ra quyết định mà không cần lập trình tường minh; phân tích mẫu trong dữ liệu lịch sử để dự đoán tương lai	Dự đoán giá nhà, lọc spam
DL (tập con của ML)	Dùng neural network **nhiều lớp** để mô hình hóa mẫu phức tạp; **tự học đặc trưng** (representation learning)	Nhận dạng ảnh, giọng nói, NLP

Định nghĩa ML cốt lõi: quá trình huấn luyện một chương trình (gọi là **model**) phân tích dataset để đưa ra dự đoán hữu ích; sau khi train, model dự đoán cho dữ liệu **mới, chưa từng thấy** (unseen data) và giúp tự động hóa quyết định.

1.2 Bản chất toán học: f và g

- Có **target function f: X → Y** (chưa biết, lý tưởng). Ta chỉ có tập huấn luyện $D = \{(x_1,y_1), \ldots, (x_n,y_n)\}$.
- Thuật toán học tìm **hypothesis function g ≈ f**. Không bao giờ có g hoàn hảo – g chỉ *xấp xỉ* f và có thể khác f.
- Vì vậy toàn bộ khóa học xoay quanh: chọn dạng g (model), chọn cách đo sai (loss), và chọn cách tìm tham số (optimizer).

1.3 ML vs. Rule-based system

Tiêu chí	Rule-based	Machine Learning
Logic	Con người viết luật cứng; kết quả xác định (**deterministic**)	Thuật toán tự học quy luật thống kê từ dữ liệu; kết quả **xác suất**
Mở rộng (scale)	Khó mở rộng; con người tham gia mọi khâu	Dễ mở rộng; cải thiện liên tục khi có dữ liệu/thuật toán tốt hơn
Dữ liệu	Ít, đơn giản	Nhiều và chất lượng tốt
Dùng khi	Dữ liệu ít, luật đơn giản, cần phản hồi nhanh, phân phối dữ liệu rõ ràng và ổn định	Luật đơn giản không đủ; môi trường biến động; phân phối dữ liệu đổi theo thời gian

1.4 Phân loại ML cơ bản (Lecture 1)

- **Supervised learning**: học từ dữ liệu có nhãn (input + output). Hai nhánh: **Regression** (đầu ra liên tục, vd giá nhà, chiều cao) và **Classification** (đầu ra rời rạc/nhãn, vd spam hay không).
- **Unsupervised learning**: chỉ có input, tự nhóm/diễn giải dữ liệu (vd **Clustering**).
- Mở rộng ở Lecture 2.0: Reinforcement, Semi-supervised, Self-supervised (xem Week 2).

1.5 Deep Learning vs. Machine Learning truyền thống

	Classic ML	Deep Learning
Phần cứng	Yêu cầu thấp, thường không cần GPU	Cao – cần GPU để tính ma trận song song trên dữ liệu lớn
Dữ liệu	Hợp với dữ liệu nhỏ; hiệu năng không tăng mãi khi thêm dữ liệu	Hiệu năng cao khi có tham số nhiều chiều và dữ liệu khổng lồ
Cách giải	Chia nhỏ bài toán từng bước	**End-to-end learning**
Feature	**Hand-crafted** (thủ công) rồi đưa vào SVM/KNN/Decision Tree	**Tự học feature** bằng thuật toán
Giải thích	Dễ giải thích	Khó giải thích

- "Deep" = số lớp của neural network; "Learning" = thuật toán học từ dữ liệu bằng cách cập nhật trọng số.
- Hai loại mạng chính: **CNN** (ảnh, "convolve" = lên mức trừu tượng cao hơn) và **RNN** (giọng nói/văn bản/âm thanh, "recur" = lặp trên chuỗi có bộ nhớ).
- Mốc lịch sử: Perceptron (1958) → Backpropagation (1974) → LeNet (1998) → **AlexNet (2012)** → VGG, GoogLeNet, ResNet (2014–2015)… Sau 2012 DL thống trị thị giác máy tính.

1.6 Neural network học như thế nào? (tóm tắt – chi tiết ở Week 3)

- Mạng gồm các lớp neuron xếp tầng, mỗi neuron có **weight** và **bias**. Thay đổi nhỏ ở weight → thay đổi nhỏ ở output.
- Học = chỉnh weight/bias để lỗi giảm, lặp lại cho tới khi phân loại đúng. **Backpropagation** tính tổng lỗi L(w) rồi truyền ngược để biết mỗi bước đóng góp bao nhiêu vào lỗi; các hàm lỗi thường dùng: MSE, SSE, Cross-Entropy (softmax).

1.7 Machine Learning Lifecycle

Vòng đời ML là chuỗi bước để xây, kiểm và triển khai model. Nó **lặp (iterative)** chứ không đi một chiều, và không có chuẩn chung duy nhất. Các khung công nghiệp: **CRISP-DM**, **SEMMA** (Sample–Explore–Modify–Model–Assess, của SAS), **TDSP** (Microsoft), **KDD**.

Pha CRISP-DM	Làm gì
1. Business Understanding	Xác định vấn đề, mục tiêu, tiêu chí thành công
2. Data Understanding	Thu thập, khám phá, đánh giá chất lượng dữ liệu
3. Data Preparation	Làm sạch, tạo feature, tích hợp, định dạng dữ liệu
4. Modeling	Chọn kỹ thuật, xây model, chỉnh tham số
5. Evaluation	Đánh giá hiệu năng, đối chiếu với mục tiêu kinh doanh
6. Deployment	Đưa model vào production (và giám sát)

Quy trình ML trong slide: Dataset → Data Collection → **Data Processing** → **Feature Extraction & Engineering** → **Feature Scaling & Selection** → Modeling (Training – Validation – Hyper-parameter tuning) → Deployment & monitoring; lặp đến khi đạt hiệu năng mong muốn.

1.8 Deep Learning framework và ví dụ MNIST

- Framework = thư viện giúp dựng model nhanh mà không phải tự cài backprop. Các tên trong slide: **TensorFlow** (Google Brain, 2015), **PyTorch** (Facebook AI Research, 2016), **Keras** (François Chollet, 2015), Theano, Torch, Caffe.
- TensorFlow: scalable (1 GPU → nhiều máy), hỗ trợ CPU/GPU, đa ngôn ngữ, đa nền tảng; TF 2.x tích hợp Keras, có TensorFlow Hub cho transfer learning.

Pipeline chuẩn của một bài toán (MNIST: nhận dạng chữ số viết tay)

Bước	Nội dung	Code TensorFlow/Keras
Data preparation	60.000 ảnh train + 10.000 ảnh test; nhãn dạng **one-hot** (vd chữ số 7 → [0,0,0,0,0,0,0,1,0,0])	—
Network definition	Softmax regression: t = W·x + b, rồi softmax → xác suất 10 lớp	`Sequential([...])`
Compile	Chọn **loss** (đo độ chính xác đầu ra), **optimizer** (cách cập nhật), **metrics** (theo dõi lúc train)	`model.compile(...)`
Train	Huấn luyện theo batch; **epoch** = 1 lượt qua toàn bộ tập train	`model.fit(...)`
Evaluate	So dự đoán với ground truth trên tập test → accuracy	`model.evaluate(...)`

1.9 Ứng dụng và Computer Vision

- Nhóm ứng dụng: **Computer Vision** (nhận dạng ảnh/khuôn mặt, object detection, ảnh y tế, OCR, lái xe tự động), **NLP** (nhận dạng giọng nói, dịch, chatbot, sentiment, tóm tắt, QA), Robotics, Generative AI, Predictive Analytics (phát hiện bất thường, đánh giá rủi ro, bảo trì dự đoán, dự báo thị trường).
- Thách thức của CV: **scale**, **viewpoint**, **illumination**, **occlusion**. Có thể kết hợp NLP + CV: image captioning, visual question answering.
- Các bài sau (W4–W6) đi sâu vào CV: classification → transfer learning → detection.

⚠ Sai lầm hay gặp – Week 01

• Nhầm "DL = ML": DL chỉ là một nhánh của ML (dùng mạng nhiều lớp), ML còn gồm SVM, Decision Tree, KNN...

• Cho rằng DL luôn tốt hơn: với dữ liệu nhỏ và bài toán đơn giản, classic ML/rule-based có thể phù hợp và rẻ hơn.

• Quên rằng g chỉ **xấp xỉ** f – đánh giá phải làm trên dữ liệu chưa thấy (test set).

W02 Algorithm Overview

Các kiểu ML · các tác vụ · bản đồ thuật toán (2.0)

i Nối với tuần trước

• Week 1 giới thiệu *khái niệm* ML; Week 2 bắt đầu *thuật toán* cụ thể. Phần 2.0 là "mục lục" cho các thuật toán sẽ học, trong đó 3 thuật toán đầu (Linear, Logistic, SVM) có bài riêng.

2.0.1 Các kiểu học máy

Kiểu	Cách học	Ví dụ
Supervised	Học từ dữ liệu có đáp án đúng	Dự đoán giá nhà từ dữ liệu cũ
Unsupervised	Không có đáp án; tìm mẫu/nhóm	Nhóm khách hàng hành vi giống nhau
Reinforcement	Thử hành động, nhận reward/penalty, cải thiện dần	Robot học điều hướng
Semi-supervised	Ít dữ liệu có nhãn + nhiều dữ liệu không nhãn. Vòng lặp: train trên dữ liệu có nhãn → dự đoán dữ liệu không nhãn → chọn dự đoán **độ tin cậy cao** → thêm vào tập train → train lại	—
Self-supervised	Model **tự tạo nhãn** từ dữ liệu	Che (mask) từ trong câu rồi dự đoán từ bị che

2.0.2 Các tác vụ (tasks)

Loại	Tác vụ
Supervised	**Regression** (dự đoán số) · **Classification** (dự đoán lớp) · **Ranking** (sắp xếp, vd kết quả tìm kiếm) · **Sequence prediction** (vd dịch máy)
Unsupervised	**Clustering** · **Association rule mining** (mua bánh mì thường mua sữa) · **Dimensionality reduction** (giảm 100 feature còn 2 để vẽ)
Reinforcement	**Decision making** (chơi game) · **Control** (điều khiển robot)

2.0.3 Bản đồ thuật toán

Thuật toán	Ý tưởng một dòng	Loss / tiêu chí	Học bằng
Linear Regression	Dự đoán số bằng hàm tuyến tính	Squared error (least squares)	Normal Eq. / GD
Logistic Regression	Dự đoán **xác suất** lớp: linear + sigmoid	Cross-entropy	GD
Decision Tree	Cây các luật chia dữ liệu	Chia để các nhóm “thuần”	Chia đệ quy
SVM	Biên phân lớp có **lề lớn nhất**	Hinge loss + ½‖w‖²	Tối ưu lề
KNN	Dựa vào K điểm giống nhất	Khoảng cách	Không có tham số học (lazy)
K-Means	Gom dữ liệu thành K cụm	Khoảng cách đến centroid	Lặp gán–cập nhật
Neural Network	Nhiều lớp unit nối nhau	Tùy bài toán	Forward + Backprop + GD

2.0.4 Ba thuật toán chỉ có ở phần 2.0

Decision Tree

- (1) Bắt đầu với toàn bộ dữ liệu → (2) tìm feature và luật **tốt nhất** để chia thành nhóm nhỏ → (3) lặp cho từng nhóm → (4) dừng khi nhóm đã **thuần** hoặc đủ nhỏ → (5) gán dự đoán cho mỗi **leaf node**.

K-Nearest Neighbours (KNN)

- Ý tưởng: input giống nhau → output giống nhau. Các bước: chọn K → tính khoảng cách từ điểm mới tới **mọi** điểm train → lấy K điểm gần nhất → dự đoán.
- **Classification**: lớp chiếm đa số trong K láng giềng. **Regression**: trung bình giá trị của K láng giềng.
- Ví dụ slide: K=3 → Class B, nhưng K=7 → Class A ⇒ **kết quả phụ thuộc K**.

K-Means Clustering

- (1) Chọn K → (2) khởi tạo ngẫu nhiên K **centroid** → (3) gán mỗi điểm vào centroid gần nhất → (4) tính lại centroid = **trung bình** các điểm được gán → (5) lặp (3)–(4) tới khi centroid ổn định.
- Đây là **unsupervised** – không dùng nhãn. Đừng nhầm K của KNN (số láng giềng) với K của K-Means (số cụm).

⚠ Sai lầm hay gặp – Week 2.0

- **KNN ≠ K-Means**: KNN là supervised (cần nhãn), K-Means là unsupervised.
- Logistic Regression có chữ "regression" nhưng là thuật toán **classification**.
- Semi-supervised ≠ Self-supervised: semi dùng một ít nhãn thật; self tự sinh nhãn từ chính dữ liệu.

W2.1 Linear Regression

Hồi quy tuyến tính · OLS · Gradient Descent · Regularization

i Nối với tuần trước

- Đây là thuật toán supervised đơn giản nhất và là **nền móng** cho mọi thứ sau: khái niệm hypothesis, cost function, Gradient Descent và regularization sẽ được dùng lại nguyên xi ở Logistic, SVM và Neural Network.

2.1.1 Bài toán và mô hình

- **Regression**: đầu ra là số thực/liên tục (tuổi, cân nặng, giá nhà). Phân loại boy/girl **không** phải regression.
- **Simple linear regression** (1 biến x): $h_\theta(x) = \theta_0 + \theta_1 x$. θ_0 = **bias/intercept**, θ_1 = **weight/coefficient**. Ví dụ slide: θ_0=0.261, θ_1=0.340 → x=15 cho y = 0.261 + 0.340·15 = 5.361.
- **Multiple linear regression** (nhiều biến): $h_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \ldots + \theta_M x_M$ với x_0=1 (M+1 tham số). Giả định các biến độc lập **không tương quan quá mạnh** với nhau.

2.1.2 Cost function: Ordinary Least Squares (OLS)

$$J(\theta_0, \theta_1) = \Sigma_{n=1..N} \left(y^{(n)} - h_\theta(x^{(n)}) \right)^2 \text{ (dạng MLR trong slide: } J(\theta) = 1/(2N) \cdot \Sigma \left(h_\theta(x^{(n)}) - y^{(n)} \right)^2)$$

- Cost này **lồi (convex)** → chỉ có **một global minimum**, không có local minimum/saddle point ⇒ GD chắc chắn hội tụ (nếu learning rate hợp lý).
- Mục tiêu: $\text{argmin}_\theta J(\theta)$. Hệ số 1/2N chỉ để đạo hàm gọn, không đổi vị trí cực tiểu.

2.1.3 Hai cách tìm tham số tối ưu

(a) Normal Equation – giải trực tiếp

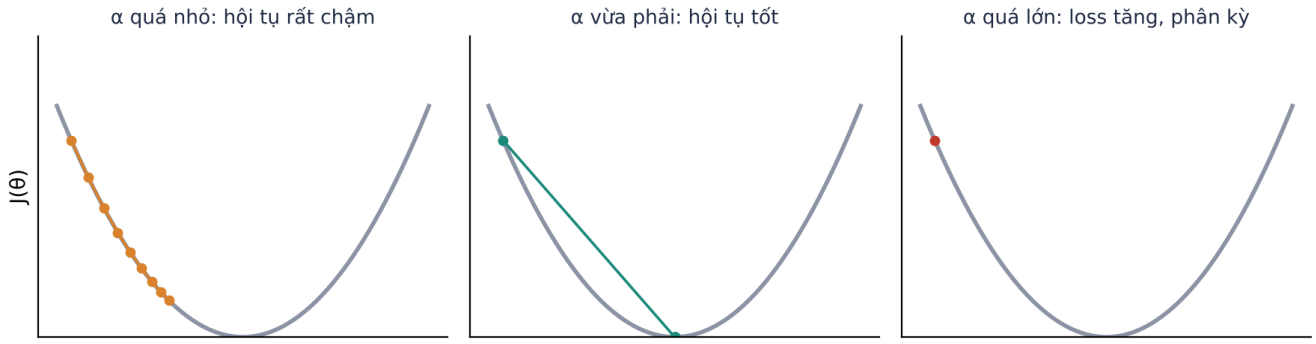
$$\theta = (X^T X)^{-1} X^T y$$

X có kích thước N × (M+1) (N mẫu, M+1 tham số, cột đầu toàn 1), y là N × 1. Suy ra từ $\partial J/\partial\theta = 0$. Cho đáp án chính xác trong **một bước**.

(b) Gradient Descent – lặp từng bước

$$\theta_j \leftarrow \theta_j - \alpha \cdot \partial J(\theta)/\partial\theta_j \text{ (cập nhật } \textbf{đồng thời} \text{ với mọi j = 0..M)}$$

- **α (learning rate)** điều khiển bước nhảy; $\partial J/\partial\theta$ là gradient chỉ hướng tăng nhanh nhất nên ta đi **ngược** chiều.
- Quy trình: khởi tạo θ ngẫu nhiên → tính cost → tính gradient → cập nhật θ → lặp đến khi cost ổn định.
- Gradient dương → θ giảm; gradient âm → θ tăng; tại minimum gradient = 0 nên θ dừng lại.
- **Cập nhật đồng thời**: tính tmp_0, tmp_1 từ θ cũ rồi mới gán cả hai. Không được cập nhật θ_0 xong rồi dùng θ_0 mới để tính θ_1.



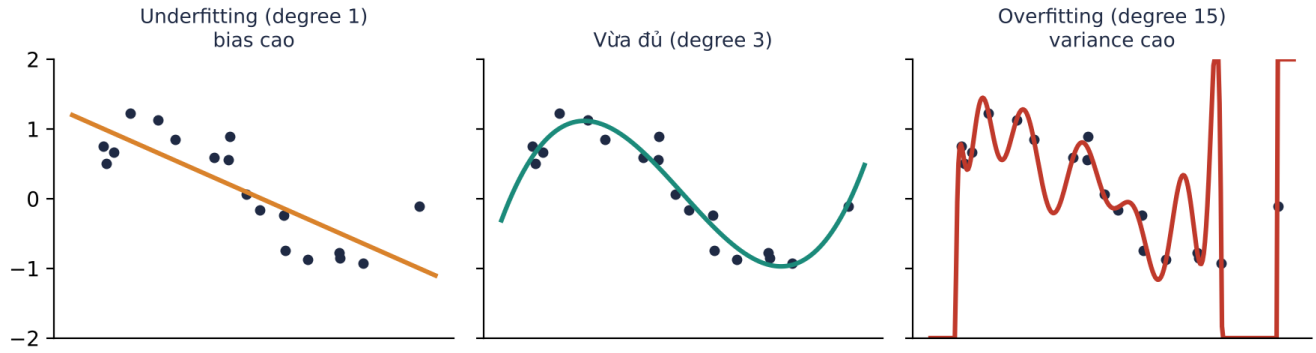
Hình 2 – Ảnh hưởng của learning rate α lên Gradient Descent (minh họa bổ sung).

	Normal Equation	Gradient Descent
Ưu điểm	Không cần chọn α; không lặp	Vẫn hiệu quả khi số tham số rất lớn
Nhược điểm	Tính $(X^TX)^{-1}$ **rất tốn** khi nhiều tham số (khoảng trăm → chục nghìn); (X^TX) có thể **không khả nghịch** nếu feature dư thừa/quá nhiều tham số	Phải chọn α; phải lặp nhiều lần

2.1.4 Polynomial Regression

- Thêm các số hạng bậc cao của x: $h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta_M x^M$ → khớp được **đường cong**.
- Vẫn là trường hợp đặc biệt của multiple linear regression vì mô hình **tuyến tính theo tham số θ** (coi x, x^2, x^3... là các feature riêng).

2.1.5 Overfitting và Underfitting



Hình 3 – Underfitting (degree 1) · vừa đủ (degree 3) · Overfitting (degree 15). Minh họa bổ sung.

	Underfitting	Overfitting
Biểu hiện	Kém trên **cả train lẫn test** (high bias, low variance)	Rất tốt trên train nhưng **tụt mạnh** trên validation/test (low bias, high variance)
Nguyên nhân	Model quá đơn giản; ít feature; train chưa đủ epoch; chọn sai model	Model quá phức tạp; dữ liệu ít; feature nhiễu/không liên quan; quá nhiều epoch
Cách xử lý	Tăng độ phức tạp (thêm layer/neuron/feature), train lâu hơn, giảm dropout	Giảm độ phức tạp/chọn feature, **regularization**, **thêm dữ liệu**

2.1.6 Regularization: L2 (Ridge) và L1 (Lasso)

$$\text{L2: } J(\theta) = 1/(2N) \cdot [\ \Sigma\ (\ h_\theta(x^{(n)}) - y^{(n)}\ )^2 + \lambda \cdot \Sigma_{j=1..M}\ \theta_j^2\ ]\ \text{L1: } \ldots + \lambda \cdot \Sigma\ |\theta_j|$$

- Thêm "hình phạt" cho weight lớn → ép θ_j nhỏ lại → model đơn giản hơn, ít nhạy với nhiễu. λ còn gọi là **weight decay**. Tổng penalty chạy từ j=1 (không phạt bias θ_0).
- **λ quá lớn** → mọi $\theta_j \approx 0$, hypothesis gần như phẳng → **underfitting**. **λ quá nhỏ** → regularization gần như không có tác dụng → vẫn overfitting.

- **L1 (Lasso)** có thể ép một số weight **đúng bằng 0** → dùng để **chọn feature**. **L2 (Ridge)** co weight đều và mượt, giữ lại mọi feature.

✎ Mẹo nhớ

• **L1 = sparse = chọn feature**; **L2 = shrink mượt = giảm overfit mà giữ mọi feature**.

• Normal Equation: "giải một phát nhưng đảo ma trận đắt"; Gradient Descent: "lặp nhiều lần nhưng scale tốt". Từ Week 3 trở đi, deep learning **luôn** dùng GD (vì không có công thức đóng).

⚠ Sai lầm hay gặp – Week 2.1

• Quên cập nhật tham số **đồng thời** trong GD.

• Chọn α quá lớn → loss tăng/phân kỳ; quá nhỏ → hội tụ cực chậm.

• Tưởng Polynomial Regression là "phi tuyến" theo tham số – thực ra vẫn tuyến tính theo θ.

• Nhầm bias θ_0 với hệ số regularization – θ_0 thường không bị phạt.

W2.2 Logistic Regression

Sigmoid · decision boundary · cross-entropy · multiclass

i Nối với tuần trước

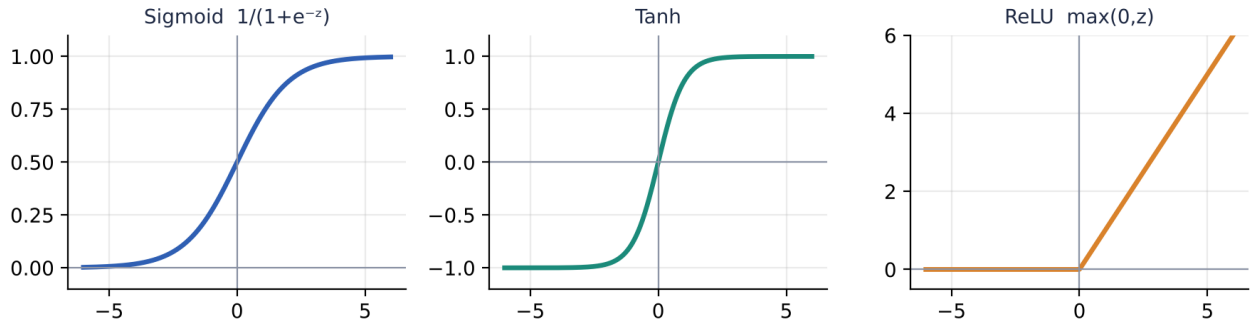
• Linear Regression dùng được cho phân loại bằng ngưỡng cứng 0.5, nhưng có nhược điểm. Logistic Regression **giữ nguyên $z = \theta^T x$** và bọc nó trong hàm **sigmoid** để đầu ra là **xác suất** ∈ [0,1]. Cost cũng đổi từ squared error sang cross-entropy. GD vẫn y như tuần trước.

2.2.1 Vì sao không dùng Linear Regression + hard threshold?

- Có thể ngưỡng hóa: y = 1 nếu $h_\theta(x) \geq 0.5$, ngược lại y = 0.
- Nhưng nhạy với **outlier và dữ liệu mất cân bằng** (đường thẳng bị kéo lệch); đầu ra **không phải xác suất** (có thể < 0 hoặc > 1).

2.2.2 Mô hình Logistic Regression

$$z = \theta^T x = \theta_0 + \theta_1 x_1 + \ldots ; \; h_\theta(x) = g(z) = 1 / (1 + e^{-z}) = P(y = 1 \mid x; \theta)$$



Hình 4 – Sigmoid, Tanh, ReLU (các activation dùng từ Week 2.2 đến Week 4).

- Sigmoid nén z ∈ (−∞, +∞) về (0, 1). Diễn giải: $h_\theta(x) = 0.91$ nghĩa là xác suất 91% bệnh nhân béo phì; P(y=0|x) = 1 − P(y=1|x).
- **Quy tắc dự đoán**: y = 1 khi $h_\theta(x) \geq 0.5 \Leftrightarrow g(z) \geq 0.5 \Leftrightarrow \mathbf{z \geq 0} \Leftrightarrow \theta^T x \geq 0$. Ngược lại y = 0.

2.2.3 Decision boundary (biên quyết định)

- Biên chia không gian feature thành các vùng lớp. Với Logistic Regression biên là tập hợp điểm thỏa **$\theta^T x = 0$** (thẳng khi 1–2 feature).
- Ví dụ slide (Iris, chiều dài đài hoa): $\theta_0 = -1.033$, $\theta_1 = 0.192$ → y=1 khi $x \geq 1.033/0.192 \approx$ **5.380**.
- Với 2 feature: $\theta_0 = -0.588$, $\theta_1 = 0.225$, $\theta_2 = -0.205$ → biên: $x_2 = (0.225/0.205) \cdot x_1 - 0.588/0.205$.
- **Biên phi tuyến**: thêm feature bậc cao (x_1^2, x_2^2, $x_1 x_2$, …) vào z để có biên cong.

	Linear Regression	Logistic Regression
Đường khớp	Đường thẳng	Đường cong chữ S (sigmoid)
Đầu ra y	Liên tục	Rời rạc (xác suất → lớp)
Cần quan hệ tuyến tính giữa biến?	Có	Không bắt buộc

2.2.4 Cost function: Binary Cross-Entropy

$$J(\theta) = -(1/N)\ \Sigma\ [\ y^{(n)} \cdot \log h_\theta(x^{(n)}) + (1 - y^{(n)}) \cdot \log(1 - h_\theta(x^{(n)}))\ ]$$

- Nếu y=1: loss = −log h; nếu y=0: loss = −log(1−h). Dự đoán **sai mà tự tin** (vd h→0 khi y=1) bị phạt cực nặng.
- Lý do không dùng squared error: sigmoid phi tuyến → J(θ) trở thành **non-convex** (nhiều local minimum). Cross-entropy cho J **lồi** với Logistic Regression.
- (bổ sung) Gradient của BCE theo θ_j có dạng rất giống Linear Regression: $(1/N)\ \Sigma\ (h_\theta(x) - y)\cdot x_j$; tối ưu bằng Gradient Descent như bình thường.

2.2.5 Multiclass: One-vs-All và Multinomial (Softmax)

	One-vs-All (OvA)	Multinomial (Softmax)
Ý tưởng	Với mỗi lớp c, train 1 bộ phân loại nhị phân "lớp c" vs "tất cả lớp khác"	Một mô hình tính điểm $z_c = \theta_c^T x$ cho mọi lớp rồi qua softmax
Dự đoán	Chọn lớp có **$\max_c\ h_\theta^{(c)}(x)$**	Chọn lớp có xác suất cao nhất
Đặc điểm	K bộ phân loại độc lập; tổng xác suất không nhất thiết = 1	Xác suất các lớp **cộng bằng 1**
Loss	BCE cho từng bộ	Categorical cross-entropy

$$\text{softmax}(z)_c = e^{z_c} / \Sigma_{i=1..K}\ e^{z_i}$$ ví dụ: [0.80, 0.15, 0.05] → chọn lớp 1 (setosa)

Cross-entropy đa lớp: $J = -(1/N)\ \Sigma_i\ \Sigma_k\ y_{ik} \cdot \log(\hat{p}_{ik})$, với $y_{ik} = 1$ nếu mẫu i thuộc lớp k (one-hot).

2.2.6 Ba biến thể Gradient Descent

Biến thể	Cập nhật khi nào	Ưu	Nhược
Batch	Sau khi duyệt **toàn bộ** tập train (1 epoch = 1 lượt)	Ít cập nhật nên hiệu quả tính toán; gradient ổn định	Phải nạp cả dataset vào bộ nhớ; chậm với dữ liệu lớn; ổn định quá có thể hội tụ sớm vào nghiệm kém
Stochastic (SGD)	Sau **mỗi mẫu**	Nhiễu giúp tổng quát hóa; cập nhật nhanh	Nhiều cập nhật → tốn tính toán; nhiễu → khó hội tụ
Mini-batch	Sau mỗi **nhóm mẫu nhỏ**	Cân bằng hai bên: cập nhật thường xuyên, ổn định hơn SGD, hiệu quả tài nguyên	Thêm hyperparameter **batch size**

Mini-batch GD là lựa chọn được ưa dùng nhất – và là cách `model.fit(..., batch_size=32)` hoạt động ở Week 1/4.

2.2.7 Logistic Regression với scikit-learn

```
from sklearn import linear_model
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

X_train, X_test, Y_train, Y_test = train_test_split(x, y, test_size=0.2, random_state=0)
clf = linear_model.LogisticRegression(C=1e40, solver='newton-cg')   # C = 1/(regularization strength)
clf.fit(X_train, Y_train)
y_pred = clf.predict(X_test)
print(accuracy_score(Y_test, y_pred), precision_score(Y_test, y_pred),
      recall_score(Y_test, y_pred), f1_score(Y_test, y_pred))
```

- `test_size=0.2`: 20% dữ liệu làm test; `random_state`: seed giúp chia dữ liệu **tái lập được**.
- **C là nghịch đảo** của độ mạnh regularization: C nhỏ → regularization mạnh (mặc định 1.0).

⚠ Sai lầm hay gặp – Week 2.2

- Dùng MSE cho Logistic Regression → cost non-convex.
- Nhầm chiều của C: ở scikit-learn **C nhỏ = regularize mạnh** (khác λ: λ lớn = mạnh).
- Nhầm OvA với Softmax: OvA là nhiều model nhị phân độc lập, Softmax là một model với xác suất tổng = 1.
- Nhầm sigmoid output với "chắc chắn": 0.51 và 0.99 đều dự đoán lớp 1 nhưng độ tin cậy rất khác.

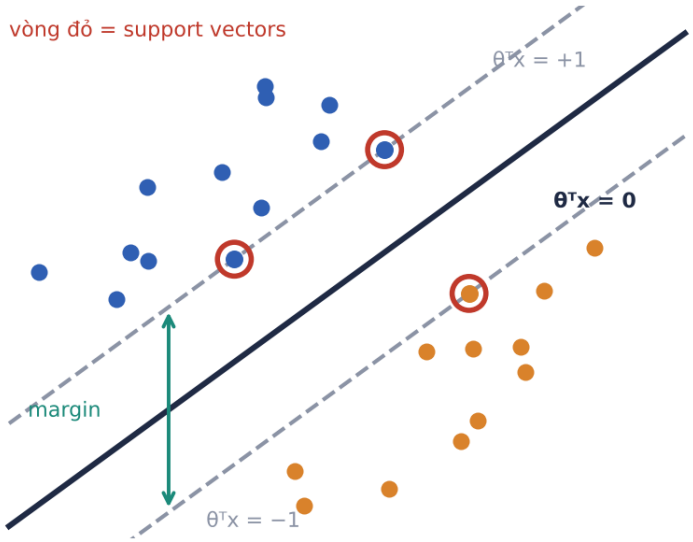
W2.3 Support Vector Machine (SVM)

Large-margin classifier · hinge loss · kernel trick

i Nối với tuần trước

- Logistic Regression có thể cho nhiều biên khác nhau đều "gần tối ưu". SVM chọn **biên duy nhất xa dữ liệu nhất** (lề lớn nhất). Loss đổi từ cross-entropy sang **hinge loss**, và C đóng vai trò giống 1/λ.

2.3.1 Ý tưởng: large-margin classifier



Hình 5 – Biên quyết định (đường đậm), lề (đường nét đứt) và support vectors (vòng đỏ).

- Mô hình: hyperplane $\mathbf{w}^T\mathbf{x} + \mathbf{b} = 0$ (hay $\theta^T x = 0$). Dự đoán: $z = w^T x + b$; $z > 0 \rightarrow$ lớp +1, ngược lại −1.
- **Margin**: khoảng cách từ biên tới điểm gần nhất mỗi lớp. **Support vectors**: chính các điểm gần nhất đó – chỉ chúng quyết định biên.
- Slide: trên mặt lề, $\theta^T x = \pm 1$; điểm giữa biên và lề có $|\theta^T x| < 1$. Khoảng cách biên → lề = $1/\|w\|$, nên **tối đa hóa margin ⇔ cực tiểu hóa $\|w\|^2$**.

2.3.2 Loss function của SVM

$$\text{Hinge loss: } L_i = \max(0, 1 - y_i \cdot (w^T x_i + b)),\ y_i \in \{-1, +1\}$$

$$J = \tfrac{1}{2} \|w\|^2 + C \cdot \Sigma_{i=1..m} L_i$$

- Điểm đúng lớp và **nằm ngoài/trên lề** → loss = 0. Điểm sai hoặc **vi phạm lề** → loss > 0, càng xa càng lớn (tăng tuyến tính).
- Số hạng 1: ½‖w‖² (tối đa hóa margin). Số hạng 2: hinge loss (giảm lỗi phân loại). Có thể viết dạng λ: $\lambda\|w\|^2 + (1/N)\ \Sigma\ L_i$, khi đó $\lambda \sim 1/C$.

	C lớn	C nhỏ
Ưu tiên	Phân loại đúng từng điểm (phạt lỗi nặng)	Lề rộng hơn, chấp nhận vài điểm sai/vi phạm lề
Rủi ro	Nhạy với **outlier** → **overfitting** (bias thấp, variance cao)	Quá nhỏ → **underfitting** (bias cao, variance thấp)
Tương đương	λ nhỏ (ít regularization)	λ lớn (nhiều regularization)

2.3.3 Dữ liệu phi tuyến và Kernel trick

- Nếu không tách tuyến tính được, **kernel function** (hàm tương tự) ánh xạ dữ liệu lên không gian chiều cao hơn, nơi có thể tách bằng hyperplane.
- Các kernel phổ biến: **Linear, Polynomial, RBF (Gaussian), Sigmoid**. Chọn kernel ảnh hưởng trực tiếp đến biên và hiệu năng.
- "Trick": máy học tuyến tính trong không gian đặc trưng chiều cao, còn độ phức tạp (capacity) được điều khiển bởi tham số không phụ thuộc số chiều.

2.3.4 SVM vs. Logistic Regression – dùng cái nào?

Tình huống (n = số mẫu, m = số feature)	Gợi ý trong slide
m rất lớn, n nhỏ (m ≫ n)	**Linear SVM** hoặc **Logistic Regression**
m nhỏ, n vừa phải (n > m)	**SVM với Gaussian (RBF) kernel**
m nhỏ, n rất lớn (n ≫ m)	**Linear SVM** hoặc **Logistic Regression** (nên thêm feature)

Khác biệt cốt lõi: SVM tìm được **margin lớn nhất**; Logistic Regression có thể có nhiều biên khác nhau với các θ gần nghiệm tối ưu.

⚠ Sai lầm hay gặp – Week 2.3

- Nhớ ngược C: **C lớn → phạt lỗi mạnh → dễ overfit**; C nhỏ → lề rộng, dễ underfit.
- Nhãn SVM là **−1/+1** (công thức hinge), không phải 0/1 như Logistic.
- Chỉ support vectors mới ảnh hưởng đến biên – bỏ các điểm khác không làm đổi model.

✎ Tổng kết nhanh Week 2

- **Linear** → số thực (MSE) · **Logistic** → xác suất (Cross-entropy) · **SVM** → lề lớn nhất (Hinge).
- Cả ba đều xuất phát từ $z = w^T x + b$; khác nhau ở **cách biến z thành đầu ra** và **loss**.
- Cả ba cost đều **lồi** ⇒ GD không bị kẹt local minimum – điều này **sẽ không còn đúng** với Neural Network (Week 3–4).

W03

Deep Neural Networks

Neuron · hidden layer · forward & backward propagation

i Nối với tuần trước

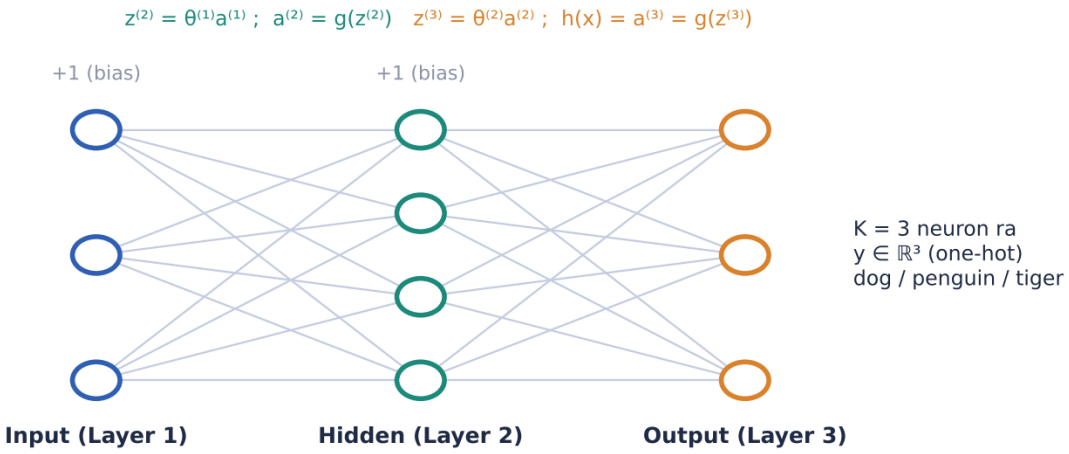
• Slide Week 3 mở đầu bằng việc chỉ ra **Logistic Regression chính là một neuron**: input x → tổng có trọng số $z = \theta^T x$ → activation sigmoid $g(z)$ → output $h_\theta(x)$. Mạng nơ-ron chỉ là xếp **nhiều** neuron như vậy thành lớp, rồi nối các lớp lại.

3.1 Vì sao cần Neural Network?

- Bắt được quan hệ **phi tuyến phức tạp**; **tự học feature** từ dữ liệu thô; tốt với dữ liệu nhiều chiều (ảnh, văn bản, âm thanh); thường đạt độ chính xác cao hơn trên bài toán khó.

3.2 Neuron và cấu trúc mạng

- **Neuron**: input x, trọng số w (parameters), **bias neuron $x_0 = 1$**, tổng có trọng số z, **activation function g**, output $a = g(z)$.
- Mạng = lớp **Input** (Layer 1) → một hoặc nhiều **Hidden layer** → lớp **Output**. Mạng sâu hơn → feature ở các lớp sau **phức tạp hơn**.
- **Logistic Regression** = mạng không hidden layer. Hidden layer chứa feature mới $a^{(2)}$ học được như hàm của $\theta^{(1)}$ và x ⇒ cho phép hypothesis phi tuyến phức tạp.



Hình 6 – Mạng 3 lớp (3-4-3) và công thức truyền thuận.

Ký hiệu	Ý nghĩa
$a_i^{(j)}$	Activation của neuron i ở lớp j
$\theta^{(j)}$	Ma trận trọng số ánh xạ từ lớp j sang lớp j+1
s_j	Số neuron ở lớp j (**không tính bias**)
Kích thước $\theta^{(j)}$	$\mathbf{s_{j+1} \times (s_j + 1)}$. Slide: 3 input, 3 hidden, 1 output → $\theta^{(1)} \in \mathbb{R}^{3\times4}$, $\theta^{(2)} \in \mathbb{R}^{1\times4}$. Mạng 3-4-3 ở Hình 6 → $\theta^{(1)} \in \mathbb{R}^{4\times4}$, $\theta^{(2)} \in \mathbb{R}^{3\times5}$
K, L	K = số neuron output; L = tổng số lớp
$\delta_j^{(l)}$	"Sai số" của neuron j ở lớp l (dùng trong backprop)

3.3 Bài toán nhiều lớp (multiclass) trong Neural Network

- Không dùng nhãn $y \in \{1,2,3\}$ mà dùng **vector one-hot** $y \in \mathbb{R}^K$: dog=[1,0,0], penguin=[0,1,0], tiger=[0,0,1]. Output $h_\theta(x) \in \mathbb{R}^K$ có K neuron, mục tiêu $h_\theta(x) \approx y$.
- Nhị phân: $y \in \{0,1\}$, $h_\theta(x) \in \mathbb{R}$ (1 neuron output).

3.4 Forward propagation (lan truyền thuận)

$$z^{(l+1)} = \theta^{(l)} a^{(l)} \; ; \; a^{(l+1)} = g(\, z^{(l+1)}\,) \; ; \; h_\theta(x) = a^{(L)}$$

Đi từ **trái sang phải**: $a^{(1)} = x$ (thêm bias) → $z^{(2)}$ → $a^{(2)}$ → $z^{(3)}$ → $a^{(3)} = h_\theta(x)$. Ví dụ slide (mạng 2-2-2-1 với n1…n5): $z_1 = w_{11}x_1 + w_{12}x_2 + b_1$, $a_1 = g(z_1)$; $z_3 = w_{31}a_1 + w_{32}a_2 + b_3$; $z_5 = w_{51}a_3 + w_{52}a_4 + b_5$.

Activation function

- g đưa **tính phi tuyến** vào mạng. Nếu bỏ g thì nhiều lớp tuyến tính chồng nhau vẫn chỉ là một hàm tuyến tính.
- **Sigmoid**: S-shape, output 0→1. **ReLU**: output 0 nếu $z \le 0$, ngược lại z (tức max(0, z)).

3.5 Backward propagation (lan truyền ngược)

Mục tiêu: tính $\partial J/\partial\theta$ cho **mọi** trọng số bằng **chain rule**, đi từ **phải sang trái**. Tại sao: weight càng ảnh hưởng nhiều tới loss thì càng cần cập nhật nhiều (“large influence → large update”).

$$\delta^{(L)} = \partial J/\partial z^{(L)} = (a^{(L)} - y) \odot g'(z^{(L)})$$

$$\delta^{(l)} = (\, \theta^{(l)}\,)^T \, \delta^{(l+1)} \odot g'(\, z^{(l)}\,) \quad \partial J/\partial \theta_{jk}^{(l)} = \delta_j^{(l+1)} \cdot a_k^{(l)}$$

- $\odot$ là nhân từng phần tử (element-wise). **Không có $\delta^{(1)}$** vì lớp input không có lỗi.
- Trực giác: forward tính tổng có trọng số từ trái sang phải; backward lan sai số δ từ phải sang trái, đo xem “nên chỉnh θ bao nhiêu để ảnh hưởng tới $h_\theta(x)$ và $J(\theta)$”.
- Ví dụ slide: $\partial L/\partial w_{51} = (\partial L/\partial a_5)(\partial a_5/\partial z_5)(\partial z_5/\partial w_{51})$; w_{31} ảnh hưởng L qua $z_3 \to a_3 \to z_5 \to L$ (dài hơn nên chain rule có nhiều thừa số).
- (bổ sung) Với sigmoid: $g'(z) = g(z)(1 - g(z))$ – tối đa chỉ 0.25, nguyên nhân gốc của vanishing gradient (xem Week 4).

Thuật toán Backpropagation (6 bước)

#	Bước	Nội dung
1	Input	Tập huấn luyện $(x^{(n)}, y^{(n)})$, n = 1..N
2	Feedforward	Với mỗi lớp: $z^{(l+1)} = \theta^{(l)}a^{(l)}$, $a^{(l+1)} = g(z^{(l+1)})$
3	Output error	Tính $\delta^{(L)}$
4	Backpropagate	Với l = L−1 … 2: tính $\delta^{(l)}$
5	Gradient	Tính gradient cho weight và bias
6	Optimize	Dùng bất kỳ biến thể gradient descent nào để cực tiểu J(θ)

3.6 Quy trình huấn luyện Neural Network

Bước	Việc cần làm
1. Initialize	Khởi tạo weight/bias bằng **giá trị ngẫu nhiên nhỏ**. Random giúp các neuron học **feature khác nhau** (nếu giống hệt nhau, chúng cập nhật y hệt – “đối xứng”)
2. Forward	Cho input qua mạng để có dự đoán
3. Loss	So dự đoán với giá trị thật để đo lỗi
4. Backprop	Tính mỗi weight đóng góp bao nhiêu vào lỗi
5. Update	Cập nhật bằng Gradient Descent
6. Repeat	Lặp nhiều vòng đến khi lỗi nhỏ/ổn định

3.7 Logistic Regression vs. Neural Network

	Logistic Regression	Neural Network
Hợp với	Dữ liệu gần **linearly separable**, bộ dữ liệu tương đối nhỏ; model đơn giản, bền	Giả thuyết **phi tuyến phức tạp**
Cost	**Lồi** → đảm bảo tìm được global minimum	**Không lồi** → có thể bị kẹt nhiều local minima / saddle point

⚠ Sai lầm hay gặp – Week 3

- Quên bias neuron khi tính kích thước ma trận: $\theta^{(j)}$ có $\mathbf{s_{j+1} \times (s_j+1)}$ chứ không phải $s_{j+1} \times s_j$.
- Khởi tạo weight toàn 0 (hoặc cùng giá trị) → các neuron không phân hóa.
- Nhầm hướng: **forward = trái → phải** (tính z, a); **backward = phải → trái** (tính δ).
- Cho rằng NN đảm bảo tìm global minimum – không đúng vì cost không lồi.

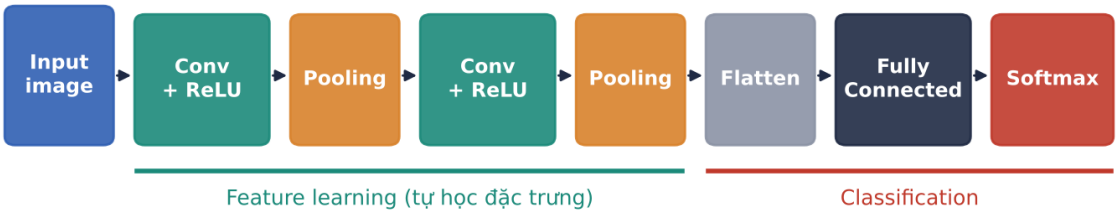
W04 Convolutional Neural Networks

Convolution · pooling · optimizer · kiến trúc · debug · Keras

i Nối với tuần trước

- (bổ sung) Một ảnh 100×100 có 10.000 pixel; nếu nối đầy đủ (fully connected) với 1.000 neuron là 10 triệu weight và bỏ qua việc pixel gần nhau liên quan chặt chẽ. CNN dùng **kết nối cục bộ + chia sẻ trọng số** để giảm tham số. Các lớp cuối của CNN (FC + softmax) chính là ANN và softmax regression đã học.

4.1 Kiến trúc CNN



Hình 7 – CNN điển hình: khối học feature (Conv + ReLU + Pooling) rồi khối phân loại (FC + Softmax).

- CNN xếp neuron theo **3 chiều** (width, height, depth); thiết kế cho dữ liệu có **tính cục bộ không gian (hoặc thời gian)**: ảnh, video, âm thanh, chuỗi thời gian. Dùng cho classification, segmentation, object detection.
- **Convolution**: tổng có trọng số trượt qua vùng cục bộ. **Pooling**: giảm kích thước bằng cách chọn giá trị đại diện.

4.2 Convolutional layer

$$x_i^{(l)} = g\left(\Sigma_{j=1..k}\, x_j^{(l-1)} * w_j^{(l-1)} + b_i \right)$$

- **Filter (kernel)** N×N trượt qua input; mỗi vị trí nhân từng phần tử rồi cộng lại, cộng bias, qua activation g → một điểm trên **feature map**. Một filter trên toàn bộ input cho một feature map.
- **N filter → N feature map**, xếp chồng thành depth của output. Ví dụ slide: input 15×15×3 với 512 filter → output 11×11×512.

Công thức kích thước output

$$\text{output} = \lfloor (d - N + 2p) / s \rfloor + 1 \quad (d: \text{input}, N: \text{filter}, p: \text{padding}, s: \text{stride})$$

Trường hợp (slide)	Tính	Output
Input 5×5, filter 3×3, stride 1, **no padding**	(5−3)/1 + 1	**3×3**
Padding = 1 (ảnh thành 7×7), filter 3×3, stride 1	(7−3)/1 + 1	**5×5** (giữ nguyên kích thước gốc)
Input 5×5, filter 3×3, **stride 2**	(5−3)/2 + 1	**2×2**

Ví dụ tính tay (khớp slide): input 5×5 = [[1,1,0,0,1],[0,1,0,1,0],[0,0,1,1,0],[1,1,0,1,1],[1,0,1,0,0]], filter = [[0,1,1],[0,1,1],[0,0,1]], stride 1 → output [[3,2,2],[2,4,3],[3,3,3]] (ô đầu: 0+1+0 + 0+1+0 + 0+0+1 = 3). **Stride** = số pixel dịch mỗi lần; **padding** = thêm viền 0.

Ba tính chất làm CNN hiệu quả

- **Weight sharing**: cùng một filter dùng cho mọi vị trí. Nếu cạnh (edge) quan trọng ở một chỗ thì cũng quan trọng ở chỗ khác ⇒ giảm mạnh số tham số và tạo **translation invariance** (nhận ra vật dù ở đâu trong ảnh).
- **Sparsity of connection**: mỗi phần tử output chỉ phụ thuộc một vùng nhỏ của input (filter 3×3 → 9 số) ⇒ cần ít mẫu hơn, bớt overfitting.
- **Local receptive field**: filter chỉ "nhìn" một mảng nhỏ.

4.3 Activation function

- **Sigmoid & Tanh** (truyền thống): bị **bão hòa** – giá trị lớn bị ép về 1 (sigmoid/tanh) hoặc nhỏ về 0/−1; chỉ nhạy quanh điểm giữa. Gây **vanishing gradient** và tính toán tốn kém. Tanh ∈ (−1, 1).
- **ReLU**: f(x) = max(0, x). Gần tuyến tính nên giữ được các tính chất giúp mô hình tuyến tính dễ tối ưu bằng gradient và khái quát hóa tốt. Tính nhanh, **giảm vanishing gradient** → mặc định cho hidden layer của CNN (AlexNet đã dùng để train nhanh hơn).

4.4 Pooling layer

- Chọn đặc trưng và **cải thiện tổng quát hóa**, giảm kích thước. Loại: **Max pooling** (phổ biến), min, average. Công thức kích thước giống convolution.
- Ví dụ slide: feature map 5×5, pool 3×3, stride 1 → 3×3 = [[8,9,9],[7,9,9],[6,6,6]]; với stride 2 → 2×2.
- Pooling **không có tham số học**. Trong Keras, `MaxPooling2D((2,2))` mặc định stride = pool size.

4.5 Fully connected layer và softmax

- Sau Flatten, **FC layer** nối mỗi neuron với **mọi** neuron lớp trước (mỗi kết nối một weight) – giống ANN. Conv layer thì chỉ nối cục bộ và dùng chung weight.
- Thay FC bằng conv layer (filter kích thước bằng feature map, rồi 1×1) giảm mạnh tham số: ví dụ slide **~81.5 triệu → ~13 triệu** và không cần Flatten.
- **Softmax** ở output đa lớp: biến vector điểm z thành phân phối xác suất, mỗi giá trị ∈ (0,1) và tổng = 1.

4.6 Optimization algorithms (vượt qua GD thường)

- Hàm loss của Linear/Logistic/SVM **lồi** nên không có saddle point; loss của NN **không lồi** và có saddle point/local minimum ⇒ SGD thường dễ kẹt vì gradient ≈ 0.

Optimizer	Ý tưởng	Trực giác
SGD + Momentum	Cộng thêm một phần cập nhật của bước trước	Dùng gradient tích lũy → nhanh hơn, giảm dao động
NAG (Nesterov)	Giống momentum nhưng tính gradient ở vị trí **phía trước**	Nhìn trước → bớt vượt quá (overshoot)
Adagrad	Learning rate thích ứng theo tham số: feature xuất hiện thường xuyên → bước nhỏ; hiếm → bước lớn	Đã cập nhật nhiều → chậm lại; hợp dữ liệu thưa
RMSprop	Chia learning rate cho trung bình trượt mũ của bình phương gradient	Thích ứng lr cho từng tham số
Adam	Kết hợp **Momentum + RMSprop**: trung bình trượt của gradient và bình phương gradient	Hiệu quả cao, **được dùng rộng rãi nhất**
Adadelta	Mở rộng Adagrad, giảm việc lr giảm đơn điệu bằng cửa sổ gradient gần nhất	Khắc phục lr co quá nhanh

4.7 Các kiến trúc CNN kinh điển

Kiến trúc	Đóng góp chính	Chi tiết cần nhớ
LeNet-5 (LeCun, 1998)	CNN đầu tiên thành công, nhận dạng chữ viết tay/in	Nguồn gốc của ý tưởng Conv → Pool → FC
AlexNet (2012)	Sâu và lớn hơn LeNet; **chồng nhiều conv** liên tiếp	ReLU thay tanh/sigmoid · nhiều GPU · overlapping pooling · **data augmentation** · **Dropout** · ~60M tham số
VGG (2014)	Chứng minh **độ sâu** là yếu tố then chốt	Thay filter lớn bằng nhiều filter **3×3** liên tiếp; khối lặp lại; VGG-16 = 13 conv + 3 FC; ~**140M** tham số, phần lớn ở FC đầu tiên
GoogLeNet (2015)	**Inception module** → giảm mạnh tham số (4M so với 60M của AlexNet)	Song song conv 1×1, 3×3, 5×5 + max pooling rồi **concatenate**; 1×1 conv để giảm chiều; mạng tự "chọn" filter size phù hợp
ResNet (He, 2016)	**Skip/residual connection**: cộng output lớp trước vào lớp sau	Giải **vanishing gradient** & **degradation problem** (mạng sâu hơn lại học kém hơn); gradient chảy thẳng qua các lớp

Hai kỹ thuật chính quy hóa/ổn định

- **Dropout**(p): trong lúc train, mỗi neuron bị **tắt** ngẫu nhiên (giữ với xác suất p, bỏ với 1−p); các kết nối vào/ra của neuron bị tắt cũng bị bỏ. Mỗi lần lặp chọn tập neuron khác. Giảm overfitting (ví dụ `Dropout(0.5)`).
- **Batch Normalization (BN)**: đặt **sau convolution, trước activation**; chuẩn hóa phân phối đầu vào của lớp để chống **internal covariate shift**, giúp train nhanh/ổn định và giảm vanishing gradient.

4.8 Quy trình thiết kế deep learning (Practical methodology)

Bước	Nội dung	Ví dụ bài biển số xe
1. Xác định mục tiêu	Bài toán gì? classification hay regression? → quyết định loss và nhãn	Vừa **regression** (vị trí biển số) vừa **classification** (ký tự)
2. Pipeline end-to-end	Chọn metric; dựng **baseline** ngay	Metric: top-1/top-5 accuracy, mAP, IoU, precision/recall; baseline: input ảnh → **CNN**; ký tự → AlexNet/VGG; detection → R-CNN family/YOLO
3. Tìm nút thắt	Overfitting? Underfitting? Lỗi dữ liệu hay phần mềm?	Quan sát learning curves
4. Cải thiện lặp đi lặp lại	Thêm dữ liệu, chỉnh hyperparameter, đổi thuật toán	—

Bốn vấn đề huấn luyện và cách xử lý

Vấn đề	Dấu hiệu / nguyên nhân	Cách khắc phục
Overfitting	Tốt trên train, kém trên test (low bias, high variance); model phức tạp, dữ liệu ít	Thêm dữ liệu; **data augmentation** (translation, flip, mirror, noise, color); **giảm** độ phức tạp mạng; L1/L2, **dropout**, **early stopping**
Underfitting	Kém cả train và test (high bias, low variance); model quá đơn giản	Tăng độ phức tạp (thêm layer/neuron); train lâu hơn; giảm dropout
Gradient exploding	Gradient tăng theo hàm mũ khi backprop → weight update khổng lồ → loss **NaN**/vô cực; hay gặp ở mạng nhiều lớp	**Gradient clipping**; giảm learning rate; weight regularization; **chuẩn hóa dữ liệu**; thiết kế mạng ít lớp hơn
Gradient vanishing	Gradient rất nhỏ → các lớp đầu gần như không được cập nhật (sigmoid bão hòa)	Dùng **ReLU**; **residual network**; **Batch Norm**; khởi tạo weight cẩn thận

Chuẩn hóa dữ liệu (Data normalisation)

- **Min-max scaling**: x_norm = $(x - x_{min}) / (x_{max} - x_{min})$ → đưa về [0,1]. Với ảnh: chia pixel cho 255.
- **Standardization**: x_norm = $(x - \mu) / \sigma$ → trung bình 0, độ lệch chuẩn 1.

Chiến lược debug

- **Định tính**: xem các vùng mô hình phát hiện (có bắt đúng biển số không?). **Định lượng**: xem các mẫu phân loại sai và xác suất softmax (vd nhầm Q với 0 có thể do nhãn sai).
- **Fit tiny data**: thử overfit trên 5–10 mẫu – nếu không được thì có bug. **Theo dõi learning curves**.

4.9 CNN với Keras

```
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),   # 32 filters 3x3
    MaxPooling2D((2,2)),                                           # default stride = pool size
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')                                # 10 classes
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',              # integer labels 3,6,2,...
              metrics=['accuracy'])                                # one-hot -> 'categorical_crossentropy'
history = model.fit(train_images, train_labels, epochs=3, validation_data=(test_images, test_labels))
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
pred = np.argmax(model.predict(images), axis=1)                    # index of the most probable class
```

- **Epoch** = một lượt qua toàn bộ dữ liệu train; **batch_size** = số mẫu mỗi lần cập nhật; **validation_data** để theo dõi overfit; `verbose` 0/1/2 = không in/thanh tiến trình/mỗi epoch một dòng.
- Với ảnh tự chuẩn bị: mở ảnh → convert RGB → resize đúng kích thước input → chuyển numpy → **chia 255** (chuẩn hóa như lúc train) → predict.

⚠ Sai lầm hay gặp – Week 4

• Tính sai kích thước output: nhớ công thức **⌊(d − N + 2p)/s⌋ + 1** và pooling dùng cùng công thức.

• Nhầm vị trí BN: **sau conv, trước ReLU**. Nhầm Dropout: chỉ bật lúc **train**.

• Chọn sai loss Keras: nhãn số nguyên → `sparse_categorical_crossentropy`; nhãn one-hot → `categorical_crossentropy`.

• Cho rằng thêm lớp luôn tốt hơn – mạng sâu gặp vanishing gradient và degradation (đó là lý do ra đời ResNet).

• Slide Week 4 ghi “Reduce complexity: increase number of neurons…” – đây là lỗi đánh máy; để giảm overfitting phải **giảm** độ phức tạp. Cũng nên viết ReLU là max(0, x), không phải max(x).

W05

Transfer Learning

Tái sử dụng tri thức · feature extractor · fine-tuning · các biến thể

i Nối với tuần trước

• Train một CNN sâu (VGG, ResNet...) từ đầu cần rất nhiều dữ liệu và thời gian. Transfer learning lấy một CNN đã học ở Week 4 (trên dữ liệu lớn như ImageNet) và **chuyển** cho bài toán mới với ít dữ liệu. Nó cũng là nền tảng cho Week 6 (base network của detector).

5.1 Transfer Learning là gì?

- Khả năng nhận ra và áp dụng tri thức/kỹ năng học từ **source task** cho **target task/domain** mới có điểm chung. Câu hỏi: nhận ra điểm chung ở đâu và chuyển gì?
- **Traditional ML**: mỗi task học riêng, không giữ lại tri thức. **Transfer learning**: học target **dựa vào** source → học nhanh hơn, chính xác hơn, cần ít dữ liệu hơn.
- Ví dụ phân loại chó: model train ở Dataset 1 kém đi khi gặp Dataset 2 (bias theo dữ liệu train). Hai chiến lược: **(1) train lại model mới** hoặc **(2) transfer learning** từ model đã có.

Mục đích

- Tận dụng model đã train · tăng tốc học · cải thiện tổng quát hóa · **giải quyết nút thắt dữ liệu** (dataset bottleneck) · giảm nhu cầu dữ liệu cho target task.

3 câu hỏi then chốt

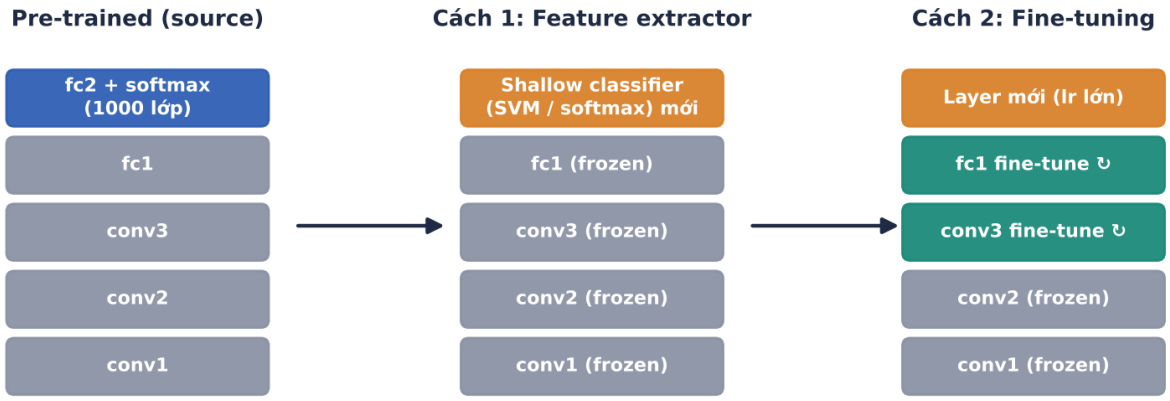
Câu hỏi	Lưu ý
What to transfer?	Phần tri thức nào chung, phần nào chỉ riêng source? Coi chừng **negative transfer** (chuyển sang làm tệ hơn). Chọn source data/model còn là bài toán mở, cần kinh nghiệm/chuyên môn
When to transfer?	Khi **không có nhiều dữ liệu target**
How to transfer?	Với deep learning: dùng pre-trained model làm feature extractor hoặc fine-tune (mục 5.3)

Quan niệm sai (myth): “không có hàng triệu mẫu có nhãn thì đừng làm deep learning”. **Thực tế**: có thể chuyển biểu diễn đã học từ task liên quan.

5.2 Domain và Task

- **Domain khác nhau** ⇔ khác **feature space** hoặc khác **phân phối biên (marginal distribution)** của dữ liệu (vd ảnh ớt chuông vs ảnh khác).
- **Task khác nhau** ⇔ khác **label space** hoặc khác **phân phối có điều kiện** P(y|x).
- Biến thể: **cùng domain, khác task** hoặc **khác domain, cùng task**.

5.3 Hai chiến lược Deep Transfer Learning chính



Hình 8 – Feature extractor (đóng băng toàn bộ, chỉ train classifier mới) và Fine-tuning (đóng băng lớp đầu, huấn luyện lại một số lớp sau).

	Off-the-shelf feature extractor	Fine-tuning
Làm gì	Dùng output của một/nhiều lớp pre-trained làm feature chung; **không cập nhật** weight (lr = 0); train một **shallow classifier** mới (SVM, random forest, softmax)	Thay lớp cuối **và** chọn lọc huấn luyện lại một số lớp trước đó; dùng trọng số pre-trained làm điểm khởi đầu
Learning rate	Lớp pre-trained: lr = 0	Lớp mới: lr_n lớn hơn; lớp pre-trained fine-tune: $lr_p > 0$ nhưng nhỏ ($lr_n > lr_p$); lớp frozen: $lr_p = 0$
Khi nào dùng	Source/target **cùng domain** và target dataset **nhỏ**	Có **nhiều dữ liệu target hơn**; muốn thích nghi tốt hơn
Rủi ro	Thích nghi kém hơn fine-tune	Overfit nếu dữ liệu ít

- **Freeze** = trọng số không được cập nhật trong backprop. **Dữ liệu ít → freeze** (tránh overfit); **dữ liệu nhiều → fine-tune**.
- Pre-trained model = bộ trọng số (hàng triệu tham số) đã được huấn luyện ổn định, chia sẻ rộng rãi. Phổ biến cho vision: **VGG-16, Inception V3 (GoogLeNet), ResNet**. Source data: **ImageNet, Places** (ảnh); Sports-1M, Kinetics, ActivityNet (video).

5.4 Các dạng Deep Transfer Learning

(a) Domain Adaptation – xử lý domain shift

Phân phối dữ liệu target khác source (**domain shift**); cần học để classifier train trên source vẫn tổng quát tốt trên target. Mục tiêu: học **domain-invariant features** (đặc trưng ít đổi giữa hai domain). Ví dụ: ảnh xe ban ngày → nhận dạng xe ban đêm; ánh sáng khác nhưng hình dạng xe, logo, bánh xe, lưới tản nhiệt không đổi.

Hướng	Ý tưởng	Chi tiết
Divergence-based	Cực tiểu hóa một tiêu chí **divergence** giữa phân phối source và target → feature bất biến miền	Divergence thường là công thức thủ công, không riêng cho task nào. Mạng gồm phần phân loại (map input → lớp) và phần ép feature hai miền giống nhau
Adversarial-based	Một **domain discriminator** cố phân biệt feature source/target; encoder cố **đánh lừa** nó	3 bước: (1) pre-train source encoder CNN với nhãn source; (2) discriminator giúp tạo feature không phân biệt được; (3) ảnh target qua target encoder vào không gian chung rồi dùng source classifier. Lúc **dự đoán bỏ discriminator**
Reconstruction-based	Image-to-image translation: encoder–decoder (+ discriminator) biến ảnh target thành ảnh "giống phong cách source"	Encoder nén ảnh target (bỏ chi tiết riêng miền) → Decoder dựng lại theo style source → **source classifier** phân loại

(b) Multi-task Learning (MTL)

- Học **đồng thời** nhiều task $t_1 \ldots t_n$ nhờ tri thức chia sẻ; không phân biệt source/target. Cấu trúc: **shared bottom layers** (biểu diễn chung) + **task-specific layers**.
- Ví dụ nhận dạng bệnh cây: ảnh lá → shared layers → nhánh "loài chủ" và nhánh "bệnh". Các feature của loài giúp suy ra bệnh.
- **Khi nào hợp lý**: các task hưởng lợi từ feature mức thấp chung; lượng dữ liệu mỗi task **tương đương**; có thể train mạng đủ lớn để làm tốt mọi task (vd DeepFashion: style, attribute, landmark).

	Transfer learning	Multi-task learning
Hướng chuyển	Source → target (một chiều: học source trước, chuyển sang target)	Các task học **cùng lúc**, chia sẻ lẫn nhau
Dữ liệu	Source lớn, target nhỏ	Mỗi task thường cỡ nhau

(c) Zero-shot learning (ZSL)

- Nhận dạng **lớp chưa từng thấy** bằng cách chuyển tri thức từ lớp đã thấy qua **thông tin phụ trợ**: thuộc tính ngữ nghĩa (semantic attributes) hoặc **word embedding**.
- Ví dụ: model biết ngựa; được mô tả "ngựa vằn là ngựa có sọc" → nhận ra ngựa vằn dù chưa train ảnh ngựa vằn.
- Cơ chế: học hàm chiếu từ **visual space** (feature ảnh) sang **semantic space** (word vector) bằng dữ liệu lớp đã thấy → ảnh lớp chưa thấy đi qua mạng ra một semantic embedding → **tìm láng giềng gần nhất** trong semantic space → nhãn tương ứng.

(d) One-shot learning

- Suy ra đầu ra chỉ với **một hoặc vài mẫu**, nhờ tri thức của các lớp đã học. Ví dụ **face recognition** (nhận diện người chỉ có 1 ảnh).
- Vấn đề của CNN thường: tập train nhỏ không đủ; thêm người/lớp mới phải **train lại** toàn bộ – quá tốn kém.
- Ý tưởng: học một **hàm tương tự (similarity function)**: mạng nhận **hai ảnh**, trả về khoảng cách $d(img_1, img_2)$. Nếu $\mathbf{d \le \lambda}$ **→ cùng đối tượng**, $d > \lambda$ → khác. Train để phân biệt cặp giống/khác rồi **tổng quát hóa sang lớp mới** (dùng distance-based loss).

⚠ Sai lầm hay gặp – Week 5

• Fine-tune toàn bộ mạng với lr lớn trên dữ liệu rất ít → phá hỏng trọng số pre-trained và overfit. Thường **freeze** lớp đầu (feature chung: cạnh, góc) và dùng lr **nhỏ** cho phần fine-tune.

• Nhầm **domain** (khác phân phối input) với **task** (khác nhãn/phân phối có điều kiện).

• Nhầm zero-shot (**không có** mẫu của lớp mới, dùng mô tả) với one-shot (**có 1–vài** mẫu).

• Quên negative transfer: nguồn quá khác target thì chuyển giao có thể hại.

• Slide Week 5 ghi "popular models that we have learn in Lecture 5" – thực ra VGG/GoogLeNet/ResNet học ở **Lecture 4**.

✎ **Mẹo nhớ**

• Ít dữ liệu + giống source → freeze hết, chỉ train classifier mới. Nhiều dữ liệu hơn → fine-tune các lớp sau.

• Lớp đầu học feature chung (cạnh, màu) → hay **freeze**; lớp sau học feature riêng task → hay **fine-tune**.

W06

Object Detection

Localization + classification · R-CNN family · YOLO · SSD

i Nối với tuần trước

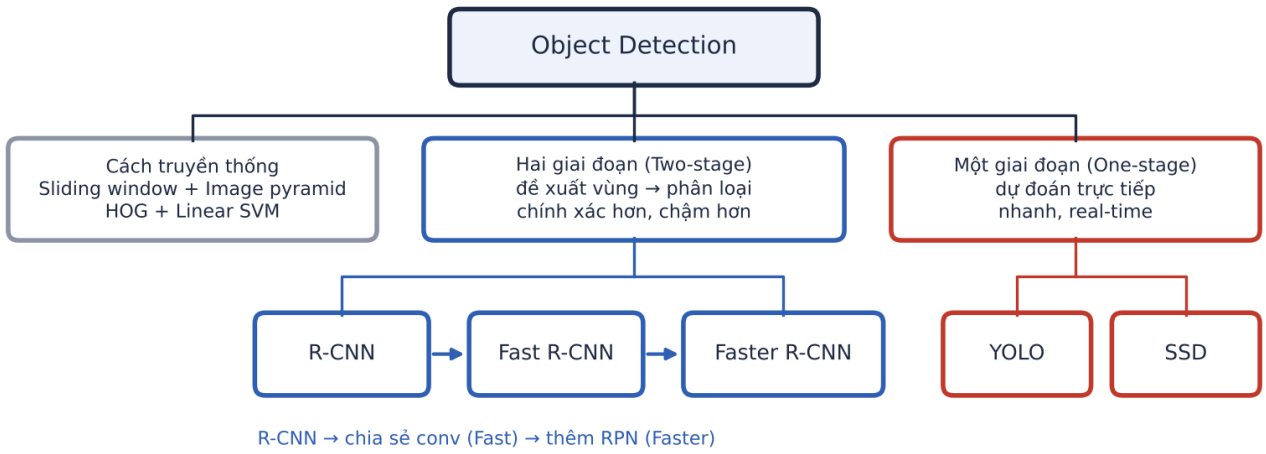
• Classification (Week 4) trả lời "ảnh này là gì?". Detection trả lời thêm "**ở đâu**, có **bao nhiêu** vật?". Các detector đều dùng CNN pre-trained (VGG, ResNet... trên ImageNet) làm **base network** – đúng tinh thần Transfer Learning của Week 5 – rồi fine-tune.

6.1 Classification vs. Localization vs. Detection

Tác vụ	Input	Output
Image classification	Ảnh	Một nhãn lớp cho cả ảnh
Object localization	Ảnh có 1 hoặc nhiều vật	Một hoặc nhiều **bounding box** (điểm + w, h)
Object detection	Ảnh có 1 hoặc nhiều vật	Bounding box **+ nhãn lớp** cho mỗi box

Vì sao CNN + FC chuẩn không dùng được?

- Số vật trong ảnh **không cố định** → độ dài output **thay đổi** (vd cat(x,y,w,h), duck(x,y,w,h), dog(x,y,w,h)...), trong khi FC cần output kích thước cố định.



Hình 9 – Phân loại các hướng tiếp cận Object Detection.

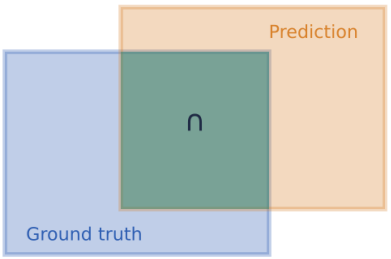
6.2 Hai cách tiếp cận

	Cách truyền thống (non-DL)	Pre-trained network làm base network
Cách làm	**Sliding window** (cửa sổ cố định trượt trái→phải, trên→dưới) + **image pyramid** (nhiều tỷ lệ) → phân loại từng **ROI** bằng CNN hoặc ML truyền thống (SVM); nếu P(L) > T thì đánh dấu box	Dùng mạng phân loại (VGG, ResNet, MobileNet, DenseNet... pre-trained trên ImageNet) làm **base network** trong framework detection (Faster R-CNN, SSD, YOLO)
Đánh giá	Chậm, rườm rà, dễ sai (vd HOG + Linear SVM)	Cần hiểu sâu cách các detector hoạt động

6.3 Two-stage vs. One-stage

	Two-stage	One-stage
Cơ chế	(1) **Đề xuất vùng** (region proposal: selective search, edge boxes hoặc mạng) → (2) bộ phân loại chỉ xử lý các vùng ứng viên	Một mạng conv **dự đoán trực tiếp** bounding box và xác suất lớp
Đại diện	R-CNN, Fast R-CNN, Faster R-CNN	YOLO, SSD
Đặc điểm	**Chính xác hơn**, nhưng chậm vì chạy dự đoán cho từng vùng	**Nhanh**, dùng cho real-time; đánh đổi một chút độ chính xác

6.4 Các độ đo và công cụ nền tảng



IoU = Area(∩) / Area(∪)

Hình 10 – IoU = diện tích giao / diện tích hợp.

- **IoU (Intersection over Union)** = Area(∩)/Area(∪) giữa box dự đoán và ground truth (0 → kém, ~0.95 → xuất sắc; slide: 0.421 poor · 0.748 good · 0.952 excellent). Dùng cả để **gán nhãn dương/âm** và **đánh giá**.
- **Precision / Recall / mAP**: mAP = trung bình AP của từng lớp. Top-1/top-5 accuracy cho phân loại.
- **Hard negative mining**: mẫu âm không đều "khó". Nền trống = "easy negative"; vùng có nhiễu hoặc một phần vật = "hard negative". Tìm các false positive khi train và đưa vào tập train để cải thiện classifier.
- **Non-Maximum Suppression (NMS)**: giữ **box tốt nhất** cho mỗi vật và **loại bỏ** các box còn lại; dựa trên **confidence score** và **IoU** giữa các box.

6.5 R-CNN (Region-based CNN)

Bước	Nội dung
1. Pre-train CNN	Train CNN phân loại (VGG/ResNet) trên ImageNet
2. Region proposal	Dùng thuật toán (Edge Boxes…) sinh ~**2k** vùng ứng viên/ảnh, nhiều kích cỡ. Vùng có IoU ≥ 0.5 với ground truth → dương, còn lại → âm
3. Fine-tune CNN	Cắt vùng, **resize/warp** về kích thước cố định; fine-tune trên **K+1 lớp** (K lớp + background). Dùng lr **nhỏ hơn** và **oversample mẫu dương** (vì đa số vùng là nền)
4. Phân loại	Mỗi vùng qua CNN một lần → vector đặc trưng → **SVM nhị phân cho từng lớp** (độc lập). Slide: dương khi IoU ≥ 0.3; dùng hard negatives
5. Bounding box regression	Hồi quy tinh chỉnh box thô → box sát ground truth, dùng feature từ CNN; box dự đoán với IoU ≥ 0.6 mới dùng để train regressor; output 4 giá trị (x, y, w, h)

- **Nhược điểm R-CNN**: **chậm lúc test** (chạy toàn bộ CNN cho mỗi proposal); **huấn luyện post-hoc** SVM/regressor nên feature CNN không được cập nhật theo chúng; **pipeline nhiều giai đoạn phức tạp** – ba model riêng (CNN, SVM, regressor) gần như không chia sẻ tính toán.

6.6 Fast R-CNN

- Hợp nhất 3 model thành **một mạng train end-to-end** và **chia sẻ tính toán conv** giữa các proposal của cùng một ảnh → giải quyết cả ba vấn đề của R-CNN.
- **ROI pooling**: một dạng max pooling đổi vùng ROI kích thước tùy ý h×w trên feature map thành cửa sổ cố định H×W – chia vùng thành lưới H×W ô (mỗi ô ≈ h/H × w/W) rồi max-pool từng ô. Đầu ra có độ dài cố định để đưa vào FC.
- Workflow: pre-train CNN → proposal (selective search, ~2k) → thay max pooling cuối bằng **ROI pooling** → mạng rẽ thành **hai nhánh**: softmax K+1 lớp và bounding-box regressor (offset cho từng lớp). Loss L = loss phân loại + loss bbox.
- **Nút thắt tốc độ**: proposal vẫn do mô hình khác tạo ra, rất tốn thời gian.

6.7 Faster R-CNN

- Chèn **Region Proposal Network (RPN)** sau lớp conv cuối để **tạo proposal trực tiếp** – không cần thuật toán ngoài; sau RPN dùng ROI pooling + classifier + box regressor như Fast R-CNN.
- **Anchor boxes**: box có kích thước và tỷ lệ khung định sẵn rải đều trên ảnh; mỗi điểm của feature map (**anchor point**) có **9** anchor (tổ hợp các kích cỡ và tỷ lệ; thường là 3 × 3). RPN cho mỗi anchor: **điểm có/không có vật** + **hồi quy** chỉnh box. Anchors **bất biến tịnh tiến** (dùng chung tại mọi vị trí).
- **Train RPN**: gán mỗi anchor một trong 3 nhãn (1, −1, 0) dựa trên IoU với ground truth; mini-batch **256** anchor từ cùng một ảnh, tỷ lệ dương:âm = **1:1** (nếu dương < 128 thì bù bằng âm); train end-to-end bằng backprop + SGD.
- **Train Faster R-CNN** (bài báo – 4 bước luân phiên): RPN → Fast R-CNN riêng → khởi tạo RPN từ Fast R-CNN và đóng băng conv chung → fine-tune lớp riêng của Fast R-CNN. Sau này dùng **joint training**: một mạng, **4 loss** (RPN cls, RPN reg, Fast R-CNN cls, Fast R-CNN reg).

6.8 YOLO (You Only Look Once)

- Detector real-time đầu tiên: **chia ảnh thành lưới S×S**; mỗi ô dự đoán **B box**, mỗi box gồm 5 giá trị (x, y, w, h, confidence) và **C xác suất lớp**. Confidence = P(object) × IoU(pred, truth).
- Output tensor: **S × S × (B·5 + C)**. Ví dụ S=7, B=2, C=20 → **7×7×30**. Backbone giống GoogLeNet (Inception thay bằng 1×1 và 3×3 conv); hai FC cuối tạo tensor dự đoán.
- Kết quả slide: **45 fps** vs 7 fps của Faster R-CNN, nhưng độ chính xác **63.4%** vs **73.2%**. Yếu với vật hình dạng bất thường hoặc **nhóm vật nhỏ** (ít box ứng viên).

6.9 SSD (Single Shot Detector)

- Dùng **VGG16** pre-trained làm base (chuyển fc6/fc7 thành conv) và thêm nhiều lớp conv **giảm dần kích thước** → dự đoán ở **nhiều tỷ lệ (multi-scale feature maps)**: map lớn/chi tiết bắt vật nhỏ; map nhỏ/thô bắt vật lớn. (YOLO chỉ dùng một scale.)
- **Convolutional predictors**: với feature map m×n×p, kernel nhỏ **3×3×p** cho điểm lớp hoặc offset so với default box. (YOLO dùng FC.)
- **Matching strategy**: dự đoán là dương nếu **default box** có IoU > 0.5 với ground truth; chỉ các match dương dùng tính localization loss.
- SSD **nhanh hơn và tốt hơn YOLO** trong slide.

Model	Loại	Điểm đổi mới	Tốc độ / độ chính xác
R-CNN	2-stage	CNN trên từng proposal + SVM + bbox regression	Rất chậm; train 3 giai đoạn
Fast R-CNN	2-stage	Chia sẻ conv, ROI pooling, train end-to-end 1 mạng	Nhanh hơn nhiều, nghẽn ở proposal ngoài
Faster R-CNN	2-stage	**RPN** + anchor boxes → hết proposal ngoài	Chính xác cao (73.2% trong slide), ~7 fps
YOLO	1-stage	Lưới S×S dự đoán trực tiếp	**45 fps**; kém hơn với vật nhỏ (63.4%)
SSD	1-stage	Multi-scale feature maps, conv predictors, default boxes	Nhanh hơn & tốt hơn YOLO

⚠ Sai lầm hay gặp – Week 6

- Nhầm **IoU ngưỡng** ở các bước R-CNN: proposal dương khi IoU ≥ 0.5 (fine-tune CNN), ≥ 0.3 (SVM theo slide), ≥ 0.6 (bbox regressor) – hãy ghi nhớ theo bước.
- Nhầm "RPN" với "selective search": RPN là **mạng** học được và chia sẻ conv; selective search/edge boxes là thuật toán ngoài.
- Nhầm ROI pooling (max pooling lên ô lưới H×W để ra output cố định) với pooling thường.
- Cho rằng one-stage luôn tốt hơn – chúng **nhanh hơn** nhưng two-stage thường chính xác hơn.
- Quên NMS: không có NMS, một vật bị nhiều box chồng nhau.

ALL

Liên kết toàn bộ kiến thức

So sánh model · concept xuyên tuần · công thức · glossary · tự kiểm tra

A. So sánh các model đã học

Model	Task	Hàm dự đoán	Loss	Điểm mạnh	Điểm yếu
Linear Reg.	Regression	$\theta^T x$	Squared error	Đơn giản; có nghiệm đóng; loss lồi	Chỉ quan hệ tuyến tính; nhạy với outlier
Logistic Reg.	Classification	$\sigma(\theta^T x)$; softmax	Cross-entropy	Ra xác suất; loss lồi; bền	Biên tuyến tính (trừ khi tự thêm feature)
SVM	Classification	$\text{sign}(w^T x+b)$; kernel	Hinge + $\frac{1}{2}\|w\|^2$	Lề lớn nhất; kernel phi tuyến	Nhạy C/kernel; chọn kernel khó
ANN	Cả hai	$g(\theta^{(l)} a^{(l)})$ nhiều lớp	CE / MSE	Tự học feature; phi tuyến	Loss không lồi; cần dữ liệu; khó giải thích
CNN	Ảnh/không gian	Conv + Pool + FC	CE	Weight sharing; ít tham số; translation invariance	Cần nhiều dữ liệu/GPU
Transfer	Strategy	CNN pre-trained + head mới	Theo task	Ít dữ liệu; train nhanh	Negative transfer; domain shift
Detector	Cls + Regression	CNN + bbox head	Cls + box regression	Biết cái gì & ở đâu	Phức tạp; cân bằng tốc độ/độ chính xác

B. Các khái niệm "xuyên tuần" – cùng một ý, xuất hiện nhiều lần

Khái niệm	Xuất hiện ở	Điểm chung / cần nhớ
Gradient Descent	W2.1 → W2.2 → W3 → W4	$\theta \leftarrow \theta - \alpha\, \partial J/\partial\theta$. W2.1 dạng cơ bản; W2.2 batch/SGD/mini-batch; W3 gradient tính bằng backprop; W4 Momentum, Adam…
Learning rate α	W2.1, W4, W5	Quá nhỏ → chậm; quá lớn → phân kỳ/exploding gradient; W5: lớp mới lr lớn hơn lớp fine-tune

Khái niệm	Xuất hiện ở	Điểm chung / cần nhớ
Sigmoid	W2.2 → W3 → W4	Logistic; neuron; rồi hạn chế (bão hòa, vanishing) → thay bằng **ReLU**
Softmax + Cross-entropy	W2.2 → W3 → W4 → W6	Đầu ra đa lớp luôn là softmax; W6 dùng softmax K+1 lớp (có background)
Overfitting / Underfitting	W2.1 → W2.3 → W4 → W5	Regularization (L1/L2, C), dropout, thêm dữ liệu, augmentation; W5: freeze khi dữ liệu ít
Regularization	W2.1 (λ) → W2.2 (C) → W2.3 (C) → W4 (dropout, BN)	Luôn là cân bằng bias–variance. λ lớn ~ C nhỏ ~ dropout nhiều
Convex vs non-convex	W2.1–2.3 → W3–4	Linear/Logistic/SVM lồi → global minimum; NN không lồi → local minima/saddle → dùng momentum/Adam
Feature learning	W1 → W3 → W4	Classic ML: feature thủ công. NN: hidden layer tự học. CNN: filter tự học (cạnh → bộ phận → vật)
Train / validation / test	W1, W2.2, W4	Đánh giá trên dữ liệu chưa thấy; `train_test_split`, `validation_data`
Pre-trained → fine-tune	W4 → W5 → W6	VGG/ResNet trên ImageNet làm base network cho detector
SVM	W2.3 → W5 → W6	W5: shallow classifier trên feature CNN; W6: R-CNN phân loại vùng bằng SVM
Regression	W2.1 → W6	Linear regression → hồi quy bounding box (x, y, w, h)
Normalization	W4 → W5/W6	Chuẩn hóa dữ liệu (min-max, standardization) và Batch Norm; đầu vào luôn phải chuẩn hóa như lúc train

C. Cheat sheet công thức

Chủ đề	Công thức
Linear model	$h_\theta(x) = \theta^T x = \theta_0 + \theta_1 x_1 + \ldots + \theta_M x_M$
Cost (MSE)	$J(\theta) = 1/(2N) \cdot \Sigma\, (h_\theta(x^{(n)}) - y^{(n)})^2$
Normal equation	$\theta = (X^T X)^{-1} X^T y$
Gradient Descent	$\theta_j \leftarrow \theta_j - \alpha\, \partial J/\partial \theta_j$ (đồng thời)
L2 / L1	$+ \lambda \Sigma\, \theta_j^2$ / $+ \lambda \Sigma\, \lvert\theta_j\rvert$
Sigmoid	$g(z) = 1/(1+e^{-z})$; dự đoán $y=1 \Leftrightarrow z \geq 0$
Binary CE	$J = -(1/N)\, \Sigma\, [y \log h + (1-y) \log(1-h)]$
Softmax	$\text{softmax}(z)_c = e^{z_c} / \Sigma\, e^{z_i}$
Hinge / SVM	$L = \max(0, 1 - y(w^T x+b))$; $J = \tfrac{1}{2}\lVert w\rVert^2 + C\, \Sigma\, L_i$
NN forward	$z^{(l+1)} = \theta^{(l)} a^{(l)}$; $a^{(l+1)} = g(z^{(l+1)})$
NN backward	$\delta^{(L)} = (a^{(L)} - y) \odot g'(z^{(L)})$; $\delta^{(l)} = (\theta^{(l)})^T \delta^{(l+1)} \odot g'(z^{(l)})$; $\partial J/\partial \theta^{(l)} = \delta^{(l+1)} a^{(l)}$
Kích thước $\theta^{(l)}$	$s_{l+1} \times (s_l + 1)$
Conv / Pool output	$\lfloor (d - N + 2p)/s \rfloor + 1$
ReLU	$\max(0, z)$
Normalization	$(x - \min)/(\max - \min) \cdot (x - \mu)/\sigma$
IoU	Area(A∩B) / Area(A∪B)
YOLO output	$S \times S \times (B \cdot 5 + C)$

D. Glossary – thuật ngữ A → Z

Thuật ngữ	Giải thích
Activation function	Hàm phi tuyến g áp lên z = tổng có trọng số (sigmoid, tanh, ReLU).
Anchor box	Box định sẵn kích thước/tỷ lệ, rải khắp ảnh; RPN hồi quy từ anchor ra proposal.
Backpropagation	Thuật toán tính gradient của loss theo mọi weight bằng chain rule, từ output về input.
Batch normalization	Chuẩn hóa đầu vào của lớp (sau conv, trước activation), chống internal covariate shift.
Bias (θ_0, b)	Hằng số dịch chuyển; cũng là thuật ngữ bias–variance (lỗi do model quá đơn giản).
Bounding box	Hình chữ nhật (x, y, w, h) bao quanh vật.
Convex	Hàm có một global minimum duy nhất.
Cross-entropy	Loss cho phân loại; phạt nặng dự đoán sai mà tự tin.
Data augmentation	Tạo thêm dữ liệu bằng biến đổi (flip, dịch, nhiễu, đổi màu) để chống overfit.
Decision boundary	Đường/mặt phân tách các lớp trong không gian feature.
Domain shift	Phân phối dữ liệu target khác source.
Dropout	Tắt ngẫu nhiên neuron lúc train để chống overfit.
Epoch / Batch	Epoch = 1 lượt qua toàn bộ dữ liệu; batch = nhóm mẫu cho một lần cập nhật.
Feature map	Kết quả áp một filter lên toàn bộ input.
Fine-tuning	Huấn luyện tiếp một số lớp của model pre-trained trên dữ liệu target.
Freeze	Không cập nhật trọng số của lớp trong lúc train.
Gradient exploding / vanishing	Gradient lớn/nhỏ theo hàm mũ khi truyền ngược qua nhiều lớp.
Hinge loss	max(0, 1 − y·z): loss của SVM.
Hyperparameter	Tham số đặt trước, không học (α, λ, C, K, batch size…).
Hypothesis (h, g)	Hàm model dùng xấp xỉ target function f.
IoU	Độ chồng lấp giữa hai box.
Kernel (SVM) / Kernel (CNN)	SVM: hàm tương tự để ánh xạ lên chiều cao; CNN: tên khác của filter.
Learning rate α	Độ lớn bước cập nhật trong gradient descent.
Margin / Support vector	Khoảng cách biên–điểm gần nhất / chính các điểm gần nhất đó.
mAP	Trung bình AP (average precision) trên các lớp.
NMS	Giữ box tốt nhất, loại box thừa chồng lấp.
One-hot	Vector nhãn toàn 0 trừ vị trí của lớp đúng = 1.
One-vs-All / Softmax	Hai cách phân loại đa lớp với Logistic Regression.
Padding / Stride	Thêm viền 0 / số pixel dịch mỗi bước của filter.
Pooling	Giảm kích thước feature map (max/average).
Pre-trained model	Model đã được huấn luyện sẵn trên dataset lớn.
Regularization	Phạt độ phức tạp (L1, L2, dropout…) để chống overfit.
ReLU	max(0, z): activation mặc định cho hidden layer.
ROI / ROI pooling	Vùng quan tâm / pooling đưa ROI bất kỳ về kích thước cố định.
RPN	Region Proposal Network – mạng sinh proposal trong Faster R-CNN.
Skip (residual) connection	Cộng output lớp trước vào lớp sau (ResNet).
Weight sharing	Dùng chung một filter trên mọi vị trí → translation invariance, ít tham số.

E. Câu hỏi tự kiểm tra

1. Viết mối quan hệ AI ⊃ ML ⊃ DL và nêu 3 khác biệt giữa classic ML và DL.

2. Vì sao g chỉ xấp xỉ f? Điều đó có hệ quả gì đến cách đánh giá model?

3. Normal Equation vs Gradient Descent: khi nào dùng cái nào? Vì sao phải cập nhật tham số đồng thời?

4. λ quá lớn / quá nhỏ trong L2 sẽ gây hiện tượng gì? Khi nào chọn L1?

5. Vì sao Logistic Regression không dùng MSE? Viết BCE và giải thích phạt "sai mà tự tin".

6. OvA và Softmax khác nhau thế nào? Batch, SGD, mini-batch khác nhau thế nào?

7. SVM chọn biên theo tiêu chí nào? C lớn và C nhỏ gây ảnh hưởng gì? Kernel trick giải quyết vấn đề gì?

8. Chứng minh/giải thích Logistic Regression là một neuron. Kích thước $\theta^{(1)}$ và $\theta^{(2)}$ của mạng 3-4-3 là bao nhiêu? (đáp án: 4×4 và 3×5)

9. Mô tả 6 bước thuật toán Backpropagation và chiều tính của δ.

10. Input 28×28, filter 5×5, stride 1, padding 0 → output? Sau MaxPool 2×2 stride 2 → output? (đáp án: 24×24 → 12×12)

11. Vì sao ReLU giảm vanishing gradient? ResNet và Batch Norm giúp gì?

12. Phân biệt vanishing và exploding gradient; mỗi loại có 3 cách xử lý.

13. Khi nào freeze, khi nào fine-tune? Khi nào có negative transfer?

14. Domain adaptation, MTL, zero-shot, one-shot khác nhau ở điểm nào?

15. Vì sao detection không dùng FC chuẩn? R-CNN có 3 vấn đề gì và Fast/Faster R-CNN giải quyết ra sao?

16. YOLO vs SSD vs Faster R-CNN: so sánh tốc độ/độ chính xác; tính output tensor của YOLO với S=7, B=2, C=20 (đáp án: 7×7×30).

17. IoU, NMS, hard negative mining dùng để làm gì?

> **i Ghi chú về nguồn**
>
> • Nội dung được tổng hợp từ 9 file slide của môn COS30082. Các mục đánh dấu "(bổ sung)" và các hình minh họa (learning rate, fitting, SVM, NN, sơ đồ roadmap) là phần giải thích thêm của tài liệu này. Các chỗ slide có lỗi đánh máy đã được nêu trong ô "Sai lầm hay gặp" của từng tuần.